

WEMA MENTAL HEALTH SCREENING MODULE

Technical Design Document | 28th June 2026

1. What We Are Building

Wema is a mental health screening tool deployed inside public hospitals. It runs on a tablet in the waiting room, after triage and before the patient sees a doctor. A patient picks it up voluntarily, completes a short evidence-based questionnaire, and is automatically routed to the right level of support based on their score. The whole flow takes under five minutes and requires no staff involvement to run.

The module integrates with Wonder HMIS so there is no separate patient registration. It pulls today's patient list from Wonder, matches the patient, and pushes screening results back — keeping clinical records in one place.

1.1 Patient Experience

Step	What Happens
Arrive & register	Patient is registered in Wonder by a CHP as normal.
Triage	Clinical triage proceeds as usual. Critical patients bypass screening entirely.
Waiting room	Stable patients are offered a tablet. Participation is voluntary.
Language & identity	Patient selects language (English / Kiswahili / Luo) and verifies identity using DOB, National ID, or phone number.
Screening	Patient selects a tool (EPDS, PHQ-2, PCL, or DAST) and completes it via text or audio narration.
Result & routing	Score is calculated instantly in the browser. Patient sees their result and next step no waiting, no staff needed.

1.2 How Results Are Routed

Risk Band	What Happens
Low (Green)	Patient receives a self-care message and returns to the main queue.
Moderate (Yellow)	System auto-books an available counsellor. Patient sees room number and name. If none are free, a queue position and estimated wait time are shown.
High (Red)	System assigns the first available psychologist. If none available: psychoeducation is shown, the psychologist WhatsApp group is notified, a CHP follow-up flag is raised, and the record is stored locally until connectivity is restored and sync completes.

2. Architecture

2.1 System Overview

The system follows a strict offline-first, sync-enabled architecture composed of four layers:

Layer	Responsibility
Browser Application (React + Vite)	Renders all screens, manages patient interaction flows, executes scoring logic locally, and reads/writes all operational data in IndexedDB via Dexie.js.
Service Worker (Workbox-based offline layer)	Caches the application shell (HTML, JS, CSS, translations, assets) and enables offline startup after first load. Intercepts network requests and serves cached assets when offline.
Synchronization Engine (Frontend module)	Manages all data movement between local storage and backend. Handles patient fetching, screening submission, retry logic, and outbox processing.
Backend API (Node.js + Express)	Provides synchronization endpoints, authentication validation, assignment logic, and integration with external systems (Wonder HMIS). Persists all canonical data in PostgreSQL.

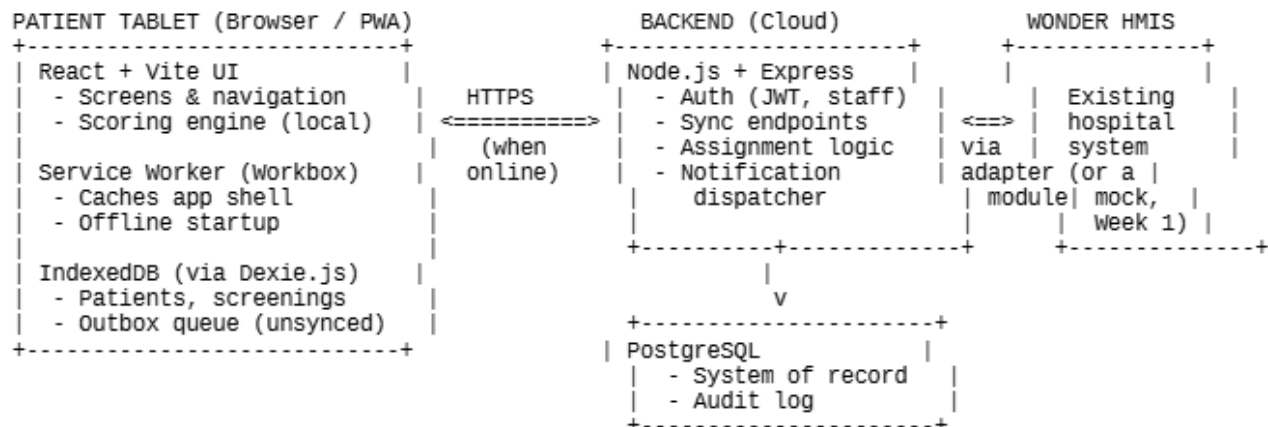
Database (PostgreSQL) acts as the system of record for all screenings, patient data, routing decisions, audit logs, and system events.

2.2 Key Architectural Principles

This system is built on five core principles:

1. Offline-first by design: All workflows execute without network dependency.
2. Local-first data writes: All user actions are stored in IndexedDB before any network interaction.
3. Sync-outbox pattern: All outbound data is queued locally before being transmitted to the backend.
4. Stateless backend: Backend does not hold session state; it only processes sync transactions and returns results.

2.3 Architecture Diagram



2.4 Offline Architecture

The application is offline-first with local execution dominance.

2.5 Offline Behavior

- All screens render from local IndexedDB state.
- Scoring runs entirely in the browser.
- No screen requires network access to function.
- All writes go to the outbox queue first.

Crash / power-loss recovery: each answer is written to IndexedDB the moment it's given, not just at the end of the questionnaire. If the app crashes, the tablet loses power, or the screen locks mid-screening, the patient's progress is not lost on reopening the app, they resume from their last answered question rather than starting over.

2.6 Service Worker Role

The Service Worker is responsible for:

- Caching the application shell after first visit.
- Enabling instant app startup on subsequent visits even offline.
- Allowing full offline UI availability.
- Serving static assets during network failure.

It does NOT:

- Handle business logic.
- Manage clinical data.
- Run scoring.
- Replace IndexedDB.

2.7 Synchronization Model

The system uses a queued sync model (outbox pattern):

1. User completes screening.
2. Record is stored in IndexedDB.
3. Record is pushed to outbox.
4. Sync engine attempts POST to backend.
5. Backend confirms + assigns routing.
6. Local record marked as synced.

Outbox pattern

Every completed screening is written to an outbox table in IndexedDB before any sync attempt. Each outbox record carries a client-generated UUID as its idempotency key. This means:

- If the POST succeeds, the record is marked synced.
- If the POST fails or times out, the record stays in the outbox and is retried on next sync.
- If the same record is POSTed twice (e.g. after a timeout where the server actually succeeded), the backend ignores the duplicate using the idempotency key.

No data is ever lost. No screening is ever double-counted.

Retry policy

- Retry uses exponential backoff: 5s, 15s, 60s, 5 minutes, then capped at 15-minute intervals so a flaky connection doesn't hammer the server.

2.8 Connectivity Behavior

State	Behavior
Online	Sync runs continuously in background.
Offline	All actions stored locally.
Reconnect	Outbox flush + patient list refresh.

3. Technology Stack

WEMA is an offline-first standard web application built with React and Vite, using IndexedDB for local storage/persistence, Service Workers for offline capability, Node.js and Express for backend services, PostgreSQL as the central database, and secure REST APIs to integrate with Wonder HMIS.

Layer	Technology	Why
Frontend	React + Vite	Modern component-based UI; fast builds. Standard web app enhanced with offline support via a service worker.
Offline/Local Storage	IndexedDB + Dexie.js	Browser's persistent database; Dexie.js simplifies querying and local transactions. Stores today's patients, screenings, sync queue, translations, config.
Offline App Shell	Service Worker + Workbox	Caches the app shell (HTML/CSS/JS) for offline startup and fast load. Asset caching only no business logic.
API Communication	Axios	Robust HTTP layer with interceptors, automatic retries, and timeout handling for all frontend–backend sync calls.
Scoring Engine	JavaScript module	Calculates PHQ-2, EPDS, PCL, and DAST scores on the tablet itself — instant, fully offline.
Backend API	Node.js + Express.js	Lightweight, fast, scalable REST APIs for sync, assignment, notifications, audit, and the Wonder adapter.
Backend Database	PostgreSQL	Central source of truth: patients, screenings, routing decisions, schedules, audit logs. Strong relational integrity.
Authentication-How does wonder authenticate	JWT	Authenticates patients. Stateless; tolerant of intermittent connectivity.
Internationalisation	react-i18next	English, Kiswahili, Luo. New languages are a content update, not a code change.
Testing (front/back/E2E)	Vitest + RTL / Jest + Supertest / Playwright	Component, API, and full end-to-end workflow testing before each release.
CI/CD	GitHub Actions	Lint + test + build on every change; auto-deploy to staging; manual promotion to production.

Layer	Technology	Why
Error Monitoring	Sentry	Captures crashes, exceptions, and sync failures essential for an unattended clinical device.
Backend Logging	Pino	Structured JSON logs for requests, sync events, and auth ready for future log aggregation.

4. Wonder HMIS Integration

All Wonder HMIS communication is isolated in a dedicated adapter module on the Express backend. Nothing in the React frontend talks to Wonder directly. This means a change to Wonder's API touches one file, not the whole codebase.

4.1 What We Need From Wonder

Requirement	Why
Daily patient list endpoint	Needed to populate the patient list sync. Without it we cannot match a patient to a Wonder record.
Counsellor availability	Needed for Moderate/High routing and wait-time estimation.
Room assignments	Needed to display a meaningful room number in routing results.
Webhooks / events	Real-time roster changes would avoid polling.

4.2 What We Need From Wonder Infrastructure & Access

Item	What we specifically need
Staging base URL	A stable staging URL (e.g. https://staging-api.wonder-hmis.example.org) we can point our adapter at during development, separate from production.
Mock or sandbox data	A seeded set of test patients and providers in staging so we aren't testing against real patient data.
API credentials	A service-to-service API key, or OAuth2 client ID/secret, scoped to staging only at first.
API documentation / OpenAPI spec	Whatever Wonder already has documented- even partial or informal docs are useful starting points.
A technical point of contact	Someone on the Wonder side who can answer integration questions and confirm response shapes during our build.